

Facade Design Pattern Real-Time Example in C#

In this article, I am going to discuss **Real-Time Examples of Facade Design Patterns using C#**. Please read our previous article where we discussed the basic concepts of **Facade Design Patterns in C#**. The Facade Design Pattern falls under the category of **Structural Design Pattern** i.e. it is going to deal with the Structural Changes of Classes, Interfaces, and the Relationship Between them.

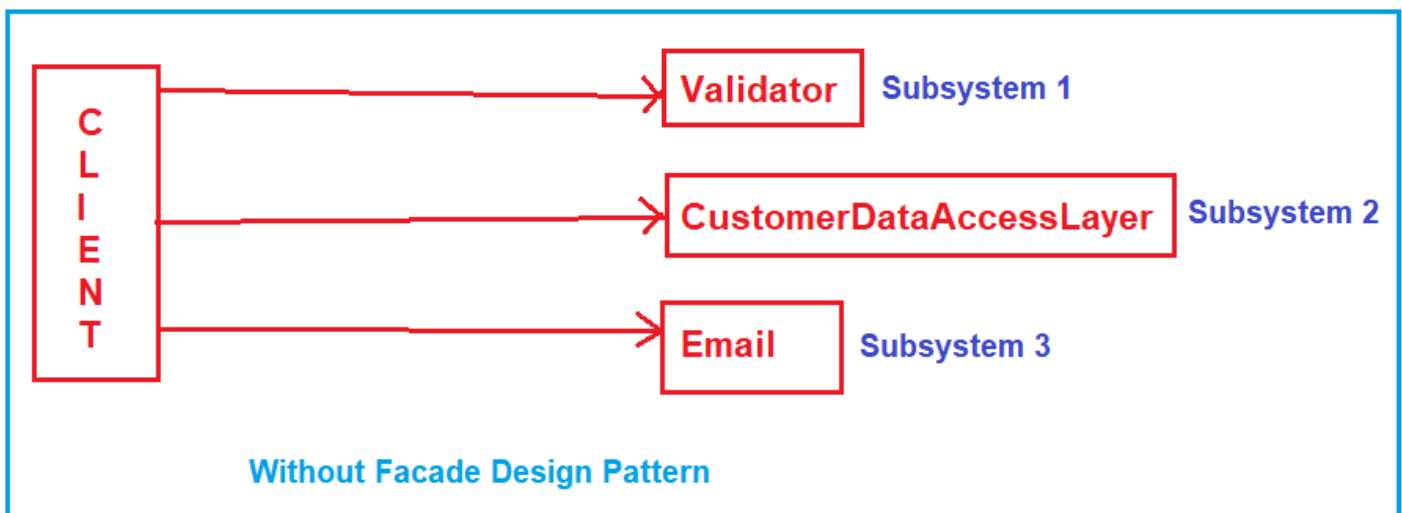
Facade Design Pattern Real-Time Example in C#

Let us understand the Facade Design Pattern with one Real-Time Example using C# Language. As we already discussed in our **previous article**, we need to use the Facade Design Pattern when the system is very complex or very difficult to understand.

Now, we are going to implement the Customer Registration Application. So, what we will do is, we will first develop the application without using the Facade Design Pattern, then we will see the problem and finally, we will see how we can overcome such problems using the Facade Design Pattern.

Example without using Facade Design Pattern in C#:

Let us first implement the Customer Registration Application without following the Facade Design Pattern. In this example, we are going to use the Customer class to store the customer data along with one class called Validator to validate the customer data, one class called CustomerDataAccessLayer to save the customer data in the database, and one class called Email to send the successful registration email to the customer. For a better understanding, please have a look at the following image which shows the CLASS Diagram without the Facade Design Pattern.



So, create a class file with the name Customer.cs and then copy and paste the following code into it. As you can see, the following Customer class contains a few properties that are required to store the customer data while doing the registration.

```
namespace FacadeDesignPatternRealTimeExample
{
    public class Customer
    {
        public string Name { get; set; }
        public string Email { get; set; }
        public string MobileNumber { get; set; }
        public string Address { get; set; }
        //Any other Properties as per the Business Requirements
    }
}
```

Next, create a class file with the name Validator.cs and then copy and paste the following code into it. As you can see, the following class contains one method called ValidateCustomer which is going to validate the Customer object.

```
using System;
namespace FacadeDesignPatternRealTimeExample
{
    public class Validator
    {
        public bool ValidateCustomer(Customer customer)
        {
            //Need to Validate the Customer Object
            Console.WriteLine("Customer Validated...");
            Console.WriteLine($"Name:{customer.Name}");
            Console.WriteLine($"Email:{customer.Email}");
            Console.WriteLine($"Mobile:{customer.MobileNumber}");
            Console.WriteLine($"Address:{customer.Address}");
            return true;
        }
    }
}
```

Note: The actual implementation for the ValidateCustomer method is not provided here. This is because we want to keep the focus on the Facade Design Pattern, not the method implementation. In real time, you need to write appropriate business logic to validate the customer data.

Once the data is validated, then we need to save the data in the corresponding database. So, create a class file with the name CustomerDataAccessLayer.cs and then copy and paste the following code into it. As you can see, the following class contains one method to save customer data in the database.

```
using System;
namespace FacadeDesignPatternRealTimeExample
{
    public class CustomerDataAccessLayer
    {
        public bool SaveCustomer(Customer customer)
        {
            //Save the Customer in the Database
            Console.WriteLine("\nCustomer Saved into the Database...");
            return true;
        }
    }
}
```

Note: The actual implementation for the SaveCustomer method is not provided here. This is because we want to keep the focus on the Facade Design Pattern, not the method implementation. In real-time, you need to write the database access logic (using ADO.NET or Entity Framework) to communicate with the database and save the data.

Once the data is saved in the database, then we need to send a successful registration email to the customer. So, create a class file with the name Email.cs and then copy and paste the following code into it. As you can see, the following class contains one method which is used to send the registration email to the customer. Like the other methods, the actual method implementation is not provided here.

```
using System;
namespace FacadeDesignPatternRealTimeExample
{
    public class Email
```

```

    {
        public bool SendRegistrationEmail(Customer customer)
        {
            //Send Registration Successful Email to Customer
            Console.WriteLine("\nRegistration Email Send to Customer...");
            return true;
        }
    }
}

```

Using the above Classes in Client Code:

Now, we need to use the above classes in the client to provide the customer registration functionality. The Client should and must use the above classes and methods in the following order.

1. First, create an instance of the Customer class, then populate the properties with the required data
2. Next, we need to create an instance of the Validator class and need to call the ValidateCustomer method by passing the customer object to validate the customer data.
3. Once the customer is validated, then the client should create an instance of the CustomerDataAccessLayer class and must call the SaveCustomer method to save the data into the database.
4. Once the data is saved successfully in the database, then the client needs to create an instance of the Email class and needs to call the SendRegistrationEmail method to send the registration successful confirmation email to the customer.

The above-mentioned steps the client needs to follow in order to provide the customer registration functionality. In our example, the Main method of the Program class is going to be the client. So, modify the Main method of the Program class as follows. Here, you can see, the client follows the above-mentioned steps and does the customer registration. The following code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
namespace FacadeDesignPatternRealTimeExample
{
    class Program
    {
        static void Main(string[] args)
        {
            //Step1: Create an Instance of Customer Class
            Customer customer = new Customer()
            {
                Name = "Pranaya",
                Email = "info@dotnettutorials.net",
                MobileNumber = "1234567890",
                Address = "BBSR, Odisha, India"
            };
            //Step2: Validate the Customer
            Validator validator = new Validator();
            bool IsValid = validator.ValidateCustomer(customer);
            //Step3: Save the Customer Object into the database
            CustomerDataAccessLayer customerDataAccessLayer = new
CustomerDataAccessLayer();
            bool IsSaved = customerDataAccessLayer.SaveCustomer(customer);
            //Step4: Send the Registration Email to the Customer
            Email email = new Email();
            email.SendRegistrationEmail(customer);
            Console.ReadKey();
        }
    }
}

```

```

    }
}
}

```

Output:

```

Customer Validated...
Name:Pranaya
Email:info@dotnettutorials.net
Mobile:1234567890
Address:BBSR, Odisha, India

Customer Saved into the Database...

Registration Email Send to Customer...

```

What is the Problem with the above Design?

Now, if you see the output, then you will see the output as expected. Then what is the problem with the above application design? The problem is now we have many subsystems like Validator, CustomerDataAccessLayer, and Email. And the Client needs to follow the appropriate sequence to create and consume the objects of the above subsystems. And here, there is a high chance that the client might not follow the proper sequence, or the client might forget to use one of the subsystems. For example, in the below code, the client forgot to use the Validator class and hence there is a chance that we might save some invalid data in the database.

```

using System;
namespace FacadeDesignPatternRealTimeExample
{
    class Program
    {
        static void Main(string[] args)
        {
            //Step1: Create an Instance of Customer Class
            Customer customer = new Customer()
            {
                Name = "Pranaya",
                Email = "info@dotnettutorials.net",
                MobileNumber = "1234567890",
                Address = "BBSR, Odisha, India"
            };

            //Step3: Save the Customer Object into the database
            CustomerDataAccessLayer customerDataAccessLayer = new
CustomerDataAccessLayer();
            bool IsSaved = customerDataAccessLayer.SaveCustomer(customer);
            //Step4: Send the Registration Email to the Customer
            Email email = new Email();
            email.SendRegistrationEmail(customer);
            Console.ReadKey();
        }
    }
}

```

Output:

```
Customer Saved into the Database...
Registration Email Send to Customer...
```

In the below example, the client uses all the subsystems but not in proper order.

```
using System;
namespace FacadeDesignPatternRealTimeExample
{
    class Program
    {
        static void Main(string[] args)
        {
            //Step1: Create an Instance of Customer Class
            Customer customer = new Customer()
            {
                Name = "Pranaya",
                Email = "info@dotnettutorials.net",
                MobileNumber = "1234567890",
                Address = "BBSR, Odisha, India"
            };
            //Step4: Send the Registration Email to the Customer
            Email email = new Email();
            email.SendRegistrationEmail(customer);
            //Step3: Save the Customer Object into the database
            CustomerDataAccessLayer customerDataAccessLayer = new
CustomerDataAccessLayer();
            bool IsSaved = customerDataAccessLayer.SaveCustomer(customer);
            //Step2: Validate the Customer
            Validator validator = new Validator();
            bool IsValid = validator.ValidateCustomer(customer);
            Console.ReadKey();
        }
    }
}
```

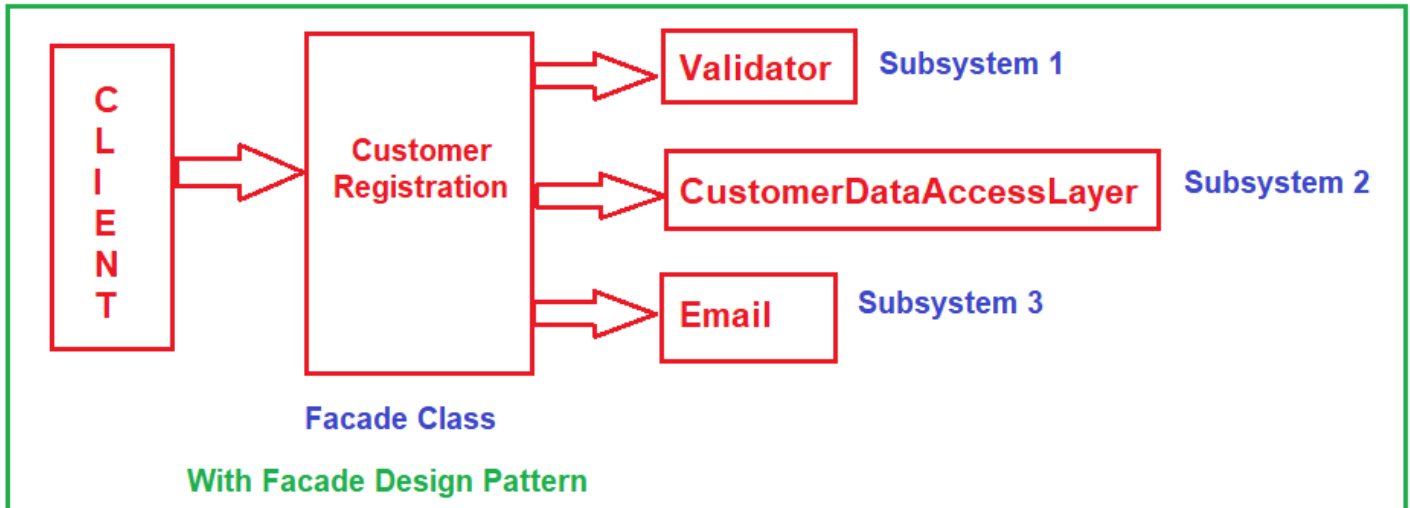
Output:

```
Registration Email Send to Customer...
Customer Saved into the Database...
Customer Validated...
Name:Pranaya
Email:info@dotnettutorials.net
Mobile:1234567890
Address:BBSR, Odisha, India
```

How we can overcome these problems?

Instead of providing access to these subsystems, if we create a simple interface and give access to the client and the client will use the simple interface to do the registration. The complex logic will be written inside the simple interface. And we can achieve this very easily by using the Facade Design Pattern in C#.

The Facade Design Pattern will hide all the complexity and provide an easy-to-use interface to the client and the client will use the Facade instead of the subsystems. So, with Facade Design Pattern, our class diagram or UML diagram will look like the one below.



Implementing the Example using Facade Design Pattern in C#:

Create a class file with the name CustomerRegistration.cs and then copy and paste the following code into it. This is going to be our Façade class. As you can see, this class contains one method called RegisterCustomer and as part of this method, first, we are validating the customer data, then we are saving the customer data into the data, and finally send the successful registration email to the customer.

```
namespace FacadeDesignPatternRealTimeExample
{
    public class CustomerRegistration
    {
        public bool RegisterCustomer(Customer customer)
        {
            //Step1: Validate the Customer
            Validator validator = new Validator();
            bool IsValid = validator.ValidateCustomer(customer);
            //Step1: Save the Customer Object into the database
            CustomerDataAccessLayer customerDataAccessLayer = new
CustomerDataAccessLayer();
            bool IsSaved = customerDataAccessLayer.SaveCustomer(customer);
            //Step3: Send the Registration Email to the Customer
            Email email = new Email();
            email.SendRegistrationEmail(customer);
            return true;
        }
    }
}
```

Now, the Client instead of using the subsystems will use the above Facade Class. So, modify the Main method of the Program class as follows. Here, you can see, the Client first creates the Customer object and then create an instance of the CustomerRegistration class and call the RegisterCustomer method by passing the customer object.

```

using System;
namespace FacadeDesignPatternRealTimeExample
{
    class Program
    {
        static void Main(string[] args)
        {
            // Create an Instance of Customer Class
            Customer customer = new Customer()
            {
                Name = "Pranaya",
                Email = "info@dotnettutorials.net",
                MobileNumber = "1234567890",
                Address = "BBSR, Odisha, India"
            };
            //Using Facade Class
            CustomerRegistration customerRegistration = new
CustomerRegistration();
            customerRegistration.RegisterCustomer(customer);
            Console.ReadKey();
        }
    }
}

```

Output:

```

Customer Validated...
Name:Pranaya
Email:info@dotnettutorials.net
Mobile:1234567890
Address:BBSR, Odisha, India

Customer Saved into the Database...

Registration Email Send to Customer...

```

In the next article, I am going to discuss the [Decorator Design Pattern in C#](#) with Examples. Here, in this article, I try to explain **Real-Time Examples of Facade Design Patterns using C#**. I hope you enjoy this Real-Time Example of Facade Design Patterns using the C# article.